

Emergency Response Mapping Sphere



Final Report

Team ERMS

Brandon Le

Kiet Nguyen

Hai Yen Nguyen

Josh Wu

Antony Quach

Department of Computer Science and Engineering
Texas A&M University

Date 4/30

Table of Contents (Be sure to regenerate before submitting).

1 Executive summary (1-2 pages; 5 points)	3
1.1 Context	4
1.2 Project Goal and Objectives	4
1.3 Final Design Implementation and Pipeline	4
1.4 Project Verification Results	5
1.5 Team/Project Management	5
2 Project background (2-4 pages; 5 points)	5
2.1 Needs statement	5
2.2 Goal and objectives	5
2.3 Design constraints and feasibility	6
2.4 Literature and technical survey	6
2.5 Evaluation of alternative solutions	8
3 Final design (5-10 pages; 5 points)	9
3.1 System description	9
3.2 Complete module-wise specifications	10
3.2.1 Hardware	10
3.2.1.1 Microcontroller Operating System Setup	10
3.2.1.2 Adafruit ISM330DHCX IMU	10
3.2.1.3 Battery Selection	11
3.2.1.4 Power Distribution Board Usage	11
3.2.1.5 3D-Printed Shell	11
3.2.2 Software	11
3.2.2.1 Data Flow	11
3.2.2.2 Gyroscope and Camera Cutting	12
3.2.2.3 Master SLAM and Gaussian Splatting	12
3.2.2.4 Object Detection	12
3.2.2.5 System User Interface	13
3.3 Approach for design validation	13
4 Implementation notes (10-20 pages; 30 points)	14
4.1 System Overview	14
4.2 Mechanical Implementation: Throwable Ball Shell	14
4.2.1 Design goals and constraints	14
4.2.2 Shell geometry and layout	15
4.2.3 Camera and IMU Standoff and Mounting	15
4.2.4 Pi Mounting and Battery Support Standoffs	16
4.2.5 3D-printing process	17
4.2.6 Mechanical Assembly Procedure	18
4.3 Electrical	20

4.4 Embedded Software	20
4.4.1 Base OS and configuration	20
4.4.2 Tailscale VPN Network Setup	21
4.4.3 Camera Capture	21
4.5 Server-side 3D Model Generation and Object Detection	21
4.5.1 Data input	21
4.5.2 Object Detection Pipeline	21
4.5.3 MAST3R-SLAM Pipeline	22
4.5.4 Web-based viewer	23
4.6 Testing	23
4.7 Future Considerations	23
5 Experimental results (5-10 pages; 20 points)	24
5.1 Physical design	24
5.2 Multi-Camera Capture	25
5.3 Gyroscope and Camera cutting	26
5.4 SLAM Modeling and YOLO Object Detection	27
5.5 Camera Performance in a low-light environment	28
5.6 End-to-End System Functionality	28
6 User's Manuals (5-10 pages; 20 points)	29
7 Course debriefing (2-4 pages; 10 points)	34
8 Budgets (2-4 pages; 5 points)	36
8.1 Detailed Cost Breakdown	36
8.2 Cost Narrative	36
8.3 Budget Comparison and Mass Production Estima	37
9 Appendices	37
10 References	37

1 Executive summary (1-2 pages; 5 points)

As you did with the proposal, the final report should start with an executive summary. In this section, you provide a brief overview of (1) the need, goal and objectives of the project, (2) the final design and implementation, (3) the results that were obtained, and (4) your experiences with team/project management. The intended audience of the executive summary is the program director at the institution/company that funded your project. Since the executive summary is a summary of the final report, it should be written last.

1.1 Context

Emergency response operations require rapid and accurate situational awareness, especially in hazardous environments where traditional methods are impractical or dangerous. First responders are routinely entering unfamiliar space such as structurally compromised buildings, active threat scenarios, and chemical spill sites, where visibility is often limited and conditions are very unpredictable. These challenges significantly increase the risk of injury, delay rescue efforts, and can lead to poor tactical decision-making in critical moments. Although current technologies such as drones, body-worn cameras, and robotic systems are used to support these operations, they frequently provide incomplete spatial information, suffer communication delays, or fail to function in confined indoor environments. Drones cannot navigate tight corridors, wheeled robots struggle with debris and rough surfaces, and stationary cameras offer limited field of vision. As a result, existing technologies do not sufficiently reduce operational risk or provide the level of real-time situational awareness necessary to ensure responder safety and mission success.

1.2 Project Goal and Objectives

The goal of this project is to build off of the previous team's version of a throwable environment scanning device capable of capturing footage of an indoor environment with camera lenses and then reconstruct a 3D mapping using Gaussian splatting. Our objective includes: implementing an IMU sensor for intelligent power management by disabling ground-facing cameras, implementing SLAM algorithm for 3D reconstruction and improving accurate capturing, object detection to classify animate vs. inanimate objects, establishing WiFi connectivity for cloud-based processing, and validating performance against metrics similar to the previous team including sub-30 second deployment-to-visualization time.

Through this project, we expect that we will be able to create 3D models accurate to at least 10% of real measurements within 30 seconds of capturing images. We also plan to be able to identify and flag objects moving faster than 20 cm/s. Through these results, first responders will obtain comprehensive 3D situational awareness within seconds of deployment, enhancing tactical decision-making in active emergencies. The throwable format requires no specialized training or line-of-sight operation. Unlike 2D feeds, 3D reconstruction enables accurate spatial judgement for planning entry strategies, identifying routes, and locating victims or threats in possible hostage situations. At under \$300 per unit for the throwable device using commercially available components, the system is substantially more affordable than specialized robotics, enabling broader deployment across agencies with limited resources. The server workstation can process multiple simultaneous deployments, supporting coordinated multi-room surveying. Beyond emergency responses, applications include building inspection, disaster search-and-rescue, and hazardous material incident response.

1.3 Final Design Implementation and Pipeline

Similar to the previous team's design, the device consists of a spherical enclosure containing 3 wide-lens No-IR cameras inside, 3 microcontrollers consisting of a Raspberry Pi 4B and 2 Pi Zero 2 W's, an Adafruit ISM330DHCX gyroscope module, a CMT Power Distribution module, a 5W 2.4A portable powerbank, and wireless networking that connects to a VPN. Once deployed, the operator can access the

web GUI to access the model output, images with object detections, and commands to enable/disable the ball wirelessly. The shell's design has been tweaked slightly from the previous team's design to incorporate elements to support our changes to the hardware.

1.4 Project Verification Results

The product passed most of our tests regarding the reliability of the ball and its physical capabilities, but failed to provide a useful model in a satisfactory amount of time, taking about ~90 seconds in realistic usage to complete a model. It was concluded that the cameras in motion would not be able to take enough suitable photos to create a sufficient model through MAST3R-SLAM. The YOLO Object Detection exceeded our expectations, however. It was able to make important detections on live objects quickly and accurately. Even being able to make important distinctions in the dark or off of partial information. Even only a shoe is enough for the system to detect a person.

1.5 Team/Project Management

Throughout the project, the team followed standard engineering project-management practices, splitting into various teams focused on different tasks to develop multiple parts of the project at once, while also meeting twice a week to share progress and collaborate on ideas on how to move forward. We kept a collection of our documentation and work on a shared google drive, while pushing all project code to our shared GitHub. We also remained in constant communication via Discord.

2 Project background (2-4 pages; 5 points)

Describe the general scope of your project, and the specific problem that your project tackles. Go from general (e.g., search and rescue robotics) to specific (e.g., GPS navigation). This section (and all its subsections) can be expanded from those in the proposal.

2.1 Needs statement

Define the specific need that your project addressed. Articulate the need as an expression of dissatisfaction with the current situation. If the needs statement was revised during the semester (e.g., because the initial needs statement may have been too restrictive), please document the changes and provide a justification.

First responders routinely have to enter dangerous structures as part of their jobs. Hazards such as structural integrity, fire, or live threats slows them down during time sensitive missions and risks their lives. From 2018 to 2020, 87% of firefighter injuries occurred during structure fires, highlighting the significant dangers associated with indoor emergency response. Additionally, active shooter incidents last an average of only 12 minutes, leaving little time for responders to assess and secure a scene. Existing solutions are upwards of thousands of dollars and tend to require a skilled operator. These statistics demonstrate the urgent need for a device that first responders can deploy quickly to enhance safety and situational awareness in hazardous environments.

The previously designed Throwable Environment Scanner addresses this need but is lacking in some areas. Currently, its 3D mapping capabilities are not clear enough to allow users to be confident that all threats have been identified, demonstrating a need for improvements in threat identification. Furthermore, users may find it difficult to use the map, showing that a more intuitive user interface is needed.

2.2 Goal and objectives

As discussed in the proposal template, the *goal* is a brief, general, and ideal response to the needs statement. The need describes the current, unsatisfactory situation; the goal describes the future condition to which we aspire. The goal statement is so ideal that it would be difficult to decide when it was

achieved. It rather establishes a general direction for the design mission. In contrast, the *objectives* are quantifiable expectations of performance. The objectives should also include a description of the conditions under which the design must perform. Again, if goal and objectives were revised during the semester, please document the changes and provide a justification for these.

The goal of this project is to enhance the Throwable Environment Scanner so that it can deliver high-quality 3D mapping directly to a first responder's device while improving existing system efficiency.

The main objective of the system is to utilize onboard cameras to stream live video and reconstruct a 3D representation of the surrounding environment, improving situational awareness and enabling more effective threat detection. At the same time, the device must maintain its portability and durability to remain practical for field deployment. It will be limited to a maximum weight of 3 pounds and a maximum diameter of 7 inches, while also being capable of withstanding drops and collisions in hazardous conditions. The device must support 30 minutes of continuous operation at a minimum to ensure reliability during active incidents and be able to start up within 30 seconds to reduce delays. Additionally, the project must remain within cost constraints, with a maximum budget of \$250 allocated for design iterations. The system must also provide threat identification in the environment to reduce mental load on users.

2.3 Design constraints and feasibility

Describe the constraints (e.g., technical, physical, economical, temporal) that you had to work with. Again, if these were revised during the semester, please document the changes and provide a justification.

One of the primary challenges of this project is ensuring that the first responders get an accurate representation of the environment with low latency. The 3D model generation and threat identification must take less time than traditional methods of ensuring safety in dangerous structures. Processing and reconstructing high-resolution spatial data requires significant computational resources, which may introduce latency and limit real-time usability. Additionally, the device must be engineered to withstand strong and repeated impacts, as it is designed to be thrown into hazardous environments. This requires a durable outer structure that protects internal components without exceeding weight and size constraints. Finally, the hardware must support complex transmission logic capable of delivering high-resolution images and mapping data quickly and reliably. Efficient data compression, wireless communication stability, and low-latency streaming will help maintain performance under emergency response conditions. Given that the durability and portability objectives of the system have already been tested, all objectives can be completed fully if threat detection and environment reconstruction algorithms are sufficiently fast and camera data quality is sufficiently accurate.

2.4 Literature and technical survey

Describe prior research and development efforts that were specifically related to your project. As stated in the proposal template, this is not meant to be a comprehensive survey of an engineering discipline, but a concise overview of the most significant results that are tightly related to your project. As a guideline, this section should include a review of *not less than five commercial products or research projects*. This review should conclude with a statement that explains the extent to which your final design is related to these other products/projects (e.g., is it better? or faster? or cheaper? does it target a new niche market?). Expand this section if additional references were found to be relevant during the semester.

While the Throwable Environment Scanner is not the first product to solve the problem of scouting hazardous indoor environments, it does improve on existing solutions by being cheaper and requiring little training to produce a 3D map. Other products might have one of these features, but none of them combine them all. By being cheap and requiring little training, it can be easily integrated into most

teams entering dangerous rooms. The 3D model visualization can allow operators to view any aspect of the environment in just a few seconds after the ball has rolled, improving decision making.

Solution	Description	Pros	Cons
The Brinc Ball	Throwable communication device that functions as a telephone	Capable of communicating with people inside dangerous area	Does not record video, scan environments, or generate maps
Recon Robotics Throwbot	Compact wheeled surveillance robot for live video and audio	- Extremely durable (30-ft drops, 120-ft throws) - Up to 60 minutes of battery life	- Requires training to operate and has only one camera angle - Obstructed terrain can limit mobility
Flyability Elios 3 (Surveying Payload)	Collision-tolerant indoor drone designed for inspection, mapping, and 3D environment reconstruction	Can actively navigate complex or obstructed spaces	Not throwable, requires skilled operation, and is high-cost industrial hardware
Bouncing Image Explorer 360	Throwable 360° tactical ball camera with live phone streaming (WiFi & Cellular)	Compact, durable, and capable of window impact deployment	No depth reconstruction (manual viewing only); high cost (\$4k–\$7k)
Squishy Robotics Stationary Robot	Throwable or aerial-deployable multi-sensor response device	Highly impact-tolerant ; designed for hazardous and disaster environments	Stationary operation; complex and high-cost system

2.5 Evaluation of alternative solutions

Discuss the alternative designs that were considered throughout the project. Analyze the pros and cons of each of these designs, and justify your decision to opt for a particular solution. You may have found additional alternatives aside from the ones in the proposal, so make sure you include all.

- **Drones**

- Pros

- Provide overhead views of environments, including inaccessible or elevated areas.
 - Already commercially available and supported by existing technology.

- Cons

- Hard to operate in enclosed areas
 - Requires significant skill from a trained operator

- **Grounded Robot**

- Pros

- Can have mobility in enclosed areas
 - A lot of them carry very sophisticated mapping and recording systems
 - Already being used by some first responders and military units

- Cons

- Mobility issues if the ground is obstructed/uneven
 - Usually large and hard to carry. Not very portable

- **Throwable Communication Ball**

- Pros

- Cheap
 - Small
 - Durable
 - Remote communication is highly useful for many areas, not limited to search and rescue, crisis intervention, disaster response, and negotiation/de-escalation

- Cons

- Cannot visualize an environment
 - Requires a conscious and willing correspondent; does not work for search and rescue of unconscious people

- **Body Cameras**

- Pros

- Cheap

- Lightweight
 - Already exists in the market
- Cons
 - Require a human in the environment, which could be hazardous
 - Remote operators cannot can only view the direction the wearer observes
- **360° Cameras**
 - Pros
 - Views the entire environment at once
 - Lightweight and portable
 - Already exists in the market
 - Cons
 - Requires a mount or transportation device to be placed somewhere

3 Final design (5-10 pages; 5 points)

3.1 System description

Please provide a description of the overall system. This description should include a high-level block diagram and a functional description of the different parts and interfaces in your final system. If any of these were revised after the CDR, please document the changes and provide a justification.

The final system is a throwable device designed to capture and reconstruct a 3D view of an environment. It is built as a spherical enclosure containing three cameras, embedded computing units (Raspberry Pi Zero 2 W and Raspberry Pi 4B), an inertial measurement unit (IMU), a battery, and a power distribution unit.

Once deployed, the cameras capture wide-angle images from different viewpoints. Each camera is connected to its own embedded unit, which handles image capture and transmits the data to a remote server. The IMU is used to determine the orientation of the device, allowing the system to identify downward-facing cameras and reduce unnecessary power consumption. The power distribution unit supplies power to all embedded components.

In the original design proposed during the Critical Design Review (CDR), a camera multiplexer was used to switch between multiple cameras on a single processor. However, this approach introduced delays due to sequential image capture. In the final design, this was replaced with multiple Raspberry Pi Zero 2 W devices and a Raspberry Pi 4B, each controlling a single camera. This allows all cameras to capture images simultaneously, improving overall system performance and image coverage.

The captured images are transmitted to a remote server through the Texas A&M IoT (TAMU IoT) network. To enable reliable and secure communication between distributed devices, the system uses Tailscale, a mesh VPN that allows each embedded unit and the server to communicate over a private network as if they were on the same local system.

On the server side, upon receiving the data, the images are synchronized and processed using 3D Gaussian Splatting to generate a 3D model of the environment. At the same time, a YOLO-based object detection model is used to identify objects within the images.

The resulting outputs, including the reconstructed 3D model and detected objects, are transmitted to an operator interface, where they can be inspected and analyzed for situational awareness and decision-making.

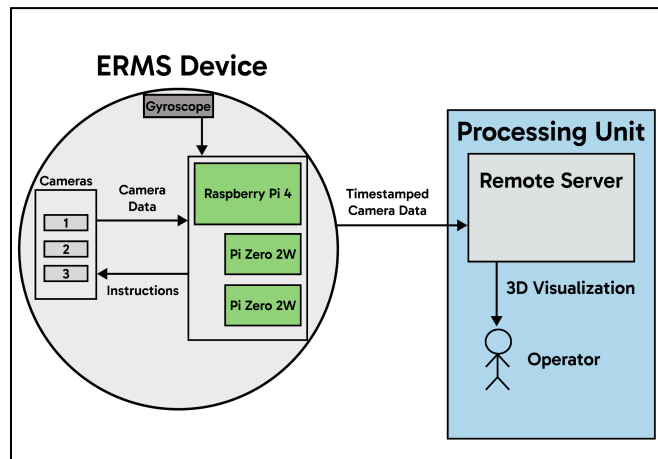


Figure 3.1.1: Final System High Level Diagram

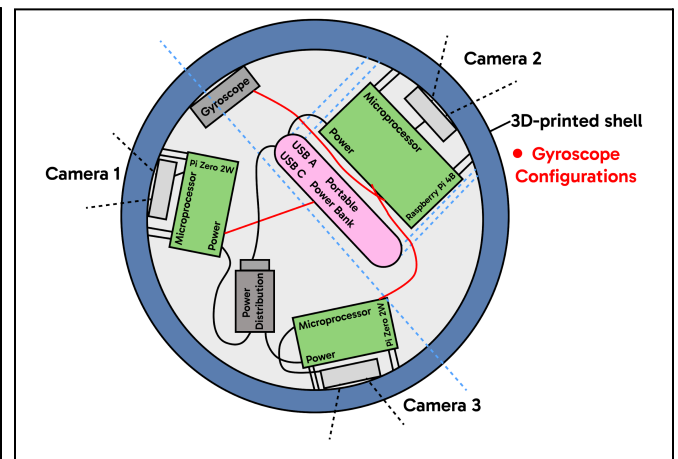


Figure 3.1.2. Internal layout of system components within the spherical enclosure

3.2 Complete module-wise specifications

Provide a detailed design of each subsystem (both hardware and software), including

- Circuit and logic diagrams
- Interfaces and pin-outs
- Timing diagrams and waveforms
- Software processes with their inputs and outputs
- Complete parts list

If any of these were revised after the CDR, please document the changes and provide a justification.

3.2.1 Hardware

3.2.1.1 Microcontroller Operating System Setup

The hardware architecture consists of three separate microcontrollers using a Raspberry Pi 4B, and two Raspberry Pi Zero 2 W's. This setup ensures parallel processing and true simultaneous camera streaming to ensure environmental capturing from multiple angles at once.

3.2.1.2 Adafruit ISM330DHCX IMU

The IMU is used to approximate the rotation of the ball to determine if any cameras are facing downward. If any cameras are detected to be facing downward, they are temporarily disabled and they do not upload images to the server. This helps save time and processing power, as images where the camera is facing downward are likely to not be useful.

3.2.1.3 Battery Selection

The powerbank we selected was a Miady 5000mAh Mini Portable Charger, 5V 2.4A USB-C. This powerbank is able to fit nicely into the shell while providing sufficient power at 5V and 2.4 A. It is also capable of powering two devices simultaneously using both of its ports.

3.2.1.4 Power Distribution Board Usage

The CMTPD Board takes an input through USB-C and regulates it to the selected voltage for output. In our case, the desired voltage output is 5V for the Raspberry Pi Zero 2W boards. We connected two micro-USB cables with pig-tailed ends directly to the output of the CMTPD Board. From there we connect the micro-USB cable to the Raspberry Pi Zero 2 W boards. This way the components are kept modular.

3.2.1.5 3D-Printed Shell

The 3D-Printed Shell houses all electronic components and shields them from external forces like heat and impacts. It has 3 camera holes equidistant from one another ensuring each camera makes full use of their 120° field of vision. It uses a series of mating flanges along the equator to secure itself together. Inside, each electronic component has its own set of standoffs where each component can be secured using M2.5 screws. The standoffs for the Raspberry Pi 4B are noticeably thicker to support the larger board. Various supports are also added to prevent the internal battery from causing damage to the Raspberry Pi 4B.

3.2.2 Software

3.2.2.1 Data Flow

The system data flow is built on a Tailscale VPN network, which establishes a secure and encrypted communication tunnel between the throwable device and the remote server. This approach enables reliable communication even when devices are connected to different Wi-Fi networks, ensuring flexibility and robustness in diverse deployment environments.

The data pipeline operates in two main stages: device-to-server data transmission and server-to-operator delivery.

On the device side, the Raspberry Pi camera units generate timestamped image frames during operation. These frames are compressed as JPEG images and downsampled to reduce bandwidth usage before being transmitted to the remote server using HTTP POST requests through a FastAPI endpoint. The server receives and stores these images in a dedicated directory, where they are monitored for downstream processing.

On the server side, incoming images are processed by two independent pipelines: a YOLO11s object detection module and a MAST3R-SLAM reconstruction pipeline. These modules operate on the same incoming dataset to generate both semantic object information and a 3D representation of the environment.

Processed outputs, including the reconstructed 3D model and annotated detection results, are served to the operator interface over HTTPS through the Tailscale network. The interface continuously polls the server for updates, enabling near real-time visualization of both spatial reconstruction and detected objects.

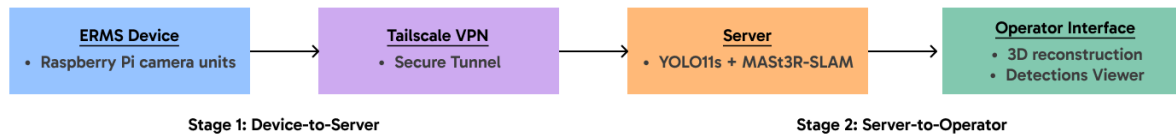


Figure 3.2.2.1: High-Level Diagram of Data Flow

3.2.2.2 Gyroscope and Camera Cutting

The purpose of this subsystem is to detect the device’s orientation in real time using a gyroscope, enabling the system to reduce unnecessary processing and improve overall power efficiency. By identifying which camera is facing downward, the system can selectively manage image capture and processing, avoiding the use of frames that are less useful for reconstruction. The implementation relies on a gyroscope that continuously streams orientation data to the main Raspberry Pi controller over an I2C interface during operation. The controller interprets this data to compute rotation updates and determine which camera is oriented downward. It then communicates this information to the other Raspberry Pi units using a GPIO-based single-bit flag. Each Pi reads this flag during image capture and uses a timing window to assess whether its camera was facing downward during the critical portion of the capture process, allowing the system to make informed decisions about which images to retain or discard.

3.2.2.3 Master SLAM and Gaussian Splatting

The MAST3R-SLAM subsystem is responsible for estimating camera motion and generating a 3D reconstruction of the environment using the captured image data. The system processes a sequence of images collected as the device moves through the environment, enabling spatial understanding of the scene.

Images are received from the capture subsystem and stored in a shared directory for processing. Using a MAST3R-SLAM implementation, the system performs feature matching across multiple images to estimate camera poses, including position and orientation for each frame. This allows the system to determine where each image was captured and reconstruct the surrounding environment.

The estimated camera poses and associated image data are then used as input to a Gaussian Splatting pipeline. This process converts sparse feature correspondences into a dense 3D representation by projecting image information into a continuous spatial model.

The reconstruction process is executed on a remote server due to the computational demands of SLAM and Gaussian Splatting. The generated 3D model is exported as a ply file and made available to the operator interface for visualization.

3.2.2.4 Object Detection

The object detection subsystem is responsible for identifying and classifying objects within the captured environment. This is implemented using the YOLO11s object detection model (YOLO “You Only Look Once” variant), which performs real-time object detection on incoming image frames. Object detection is carried out on the remote server using the same images processed by the MAST3R-SLAM pipeline, ensuring consistency between spatial reconstruction and semantic understanding of the scene.

As illustrated in **Figure 3.2.2.4** below, the object detection pipeline begins with image capture on the Raspberry Pi camera units. The captured images are transmitted to an Image Receiver API on the remote server via HTTP POST requests, where they are stored in a designated incoming images directory.

Rather than images being directly forwarded, the YOLO processing server continuously monitors this directory for newly added images. When new images are detected, the server automatically processes them and performs object detection using the YOLO11s model. The model generates bounding boxes, class labels, and confidence scores for all detected objects in each frame.

All detections produced by the model are preserved in the server-side output. However, a higher-level filtering step is applied at the image level before visualization. Only images containing at least one detected living entity, such as a person or animal, are forwarded to the detection viewer. This design reduces visual clutter by preventing frames without relevant entities from being displayed, while still retaining full detection results on the server for logging and analysis.



Figure 3.2.2.4: YOLO Object Detection Pipeline

3.2.2.5 System User Interface

The system provides a web-based interface for control and visualization of the captured and processed data. The interface is designed to be user-friendly and requires minimal technical expertise, allowing operators to interact with the system through a standard web browser.

Through this interface, a navigation bar at the top allows the user to start or stop the Raspberry Pi camera client using a single button, simplifying system operation. A real-time status indicator is also included to display the current system state, such as loading, ready, or actively processing data during 3D reconstruction.

In addition to control functionality, the viewer displays processed outputs from the backend pipeline. This includes the 3D reconstruction generated by the MAST3R-SLAM and Gaussian Splatting pipeline, as well as object detection results produced by the YOLO11s model, which are accessible in a separate detection tab. The detection results are rendered with bounding boxes and class labels overlaid on the corresponding images.

This enables simultaneous visualization of both spatial structure and semantic information, allowing operators to inspect the reconstructed environment while identifying detected objects in real time.

The interface is accessible through any standard web browser and communicates with the backend system over the Tailscale network, requiring no additional software installation beyond network access.

3.3 Approach for design validation

How did you test your system to ensure that it did what it was designed to do? If these validation procedures were revised during the semester, please document the changes and provide a justification.

- **Camera Quality:** The images taken by the camera should be checked to ensure that they clearly picture the environment and objects in the images are easy to identify. The photos captured by the camera will be compared to the environment to ensure accuracy.
- **Model Quality:** The 3D model created by MAST3R-SLAM is based off of the images collected by the ball and thus the environment it was thrown in. The model created by the server will also be compared to the environment to ensure accuracy. The generation will also be timed as time taken is also an important factor.
- **Wireless transmission:** Because the ball is intended to be used remotely, the ball must be capable of transmitting wirelessly from a distance through walls and objects. To test this, we plan to operate the ball through 10m of obstacles and evaluate for latency and data loss.

- **User interface:** Because our tool is meant to have little operator training required, the user interface must be intuitive and easy to use. To test this, we will use third parties as operators. While the test operators use the system, we will observe them and ask about their experience.
- **Mapping algorithms:** If the 3D environments are not accurate to the real environment, operators may make decisions based on incorrect information. We will compare the generated environments against measurements of the real one to ensure accuracy.
- **Live object identification:** Hidden live objects may not be easy for operators to identify. To ensure the system can identify live objects, we will operate it in an environment with humans and or animals, in varying degrees of hiddenness, and compare the generated flagged objects to the real ones to ensure that the system does accurately flag live or moving objects.
- **Lifespan test:** The device must operate for long enough that it can be used to explore multiple environments and to monitor changes in environments. The ball will be fully charged and operated until it runs out of battery to determine its lifespan.

4 Implementation notes (10-20 pages; 30 points)

This is the single most important section in the final report. It should contain “engineer's notes” that would allow a reasonably skilled engineer to understand, reproduce and modify your group’s products.

Remember that figures should include labels and captions and be referenced directly in text. An example of a figure with a label and caption can be seen in Figure 1. Additionally, tables should also be labeled and captioned and a direct in text reference. Table 1 shows an example of a labeled and captioned table. Labeling and caption may be painful to do in Google Docs as was done here, but it should be easier in MS Word, and significantly easier in LaTeX.

4.1 System Overview

At a high level, the system consists of three tightly coupled subsystems:

1. **Throwable sensor ball** – a 3D-printed spherical enclosure that houses 3 wide-lens No-IR cameras inside, 3 microcontrollers consisting of a Raspberry Pi 4B and 2 Pi Zero 2 W's, an Adafruit ISM330DHCX gyroscope module, a CMT Power Distribution module, a 5W 2.4A portable powerbank, and wireless networking that connects to a VPN. All electronics are rigidly mounted to survive impact when the ball is thrown into a room.
2. **Network and control plane** – each Pi automatically joins a Tailscale VPN and exposes a REST API and image stream to a central processing server. Time-stamped frames and metadata are transmitted over Wi-Fi.
3. **Reconstruction and visualization server** – a high-performance cloud server with an NVIDIA GPU that receives images, scans images for relevant objects, and reconstructs the environment using Gaussian-based methods. The resulting images and 3D model are uploaded to a web viewer where the results can be viewed freely and the ball can be interacted with remotely.

Refer to **Figure 3.1.1** and **Figure 3.1.2** on page 10 for a visual description.

4.2 Mechanical Implementation: Throwable Ball Shell

4.2.1 Design goals and constraints

The shell needs to:

- Protect internal hardware from external impacts and fire damage
- Reliably hold each internal component in place. The gyroscope especially must be kept in one spot relative to the shell so that it can detect the rotation of the ball correctly

- Provide unobstructed fields of view for three wide-angle cameras with minimal occlusion from the shell
- Must be easy to maintain, using standard components, and the inside should stay accessible
- Must be secure enough to stay closed even under direct impacts
- Stay within size and mass limits to preserve throwability (≤ 7 inches diameter and ≤ 1 pound weight)

These constraints drove the geometry of the shell, internal standoffs, and mounting surfaces.

4.2.2 Shell geometry and layout

The outer shell is a two-piece sphere printed from ABS 3D printing filament. ABS was chosen for its combination of impact resistance, moderate flexibility, and good heat resistance, which protects the electronics from both mechanical shocks and self-heating.

- Outer diameter: ~ 160 mm
- Wall thickness: ~ 5 mm (varies slightly with local features and standoffs)
- Joining method: mating flanges along the equator with a screw pattern and alignment features so the two halves always close in the same orientation.

Three circular camera windows are placed along the equator at 120° intervals. Each window is backed by a planar “cut” into the sphere: locally, the spherical surface is replaced by a flat patch where the camera PCB and lens mount. These tangent planes simplify alignment and ensure each camera looks roughly outward along a direction 120° apart from the others. This retains the same external design from the previous team.

4.2.3 Camera and IMU Standoff and Mounting

Each Raspberry Pi Camera Module 3 Wide has a rectangular PCB ($\sim 25 \times 24$ mm) with four mounting holes near the corners. We designed matching standoffs directly into each planar camera patch:

- Standoff layout: rectangle matching the camera PCB hole pattern (roughly $18\text{--}20$ mm $\times$ $18\text{--}20$ mm, measured from the actual board).
- Standoff outer diameter: 4.75 mm.
- Pilot hole diameter: 2.2–2.3 mm, sized for M2.5 self-tapping or machine screws.
- Height: enough to bring the PCB flush with the planar surface while keeping the lens protruding slightly beyond the outer surface, ensuring the glass is not recessed where it could see the edge of the shell.

The Adafruit ST ISM330DHCX IMU also needs to be secured in one spot relative to the ball

- The IMU’s mounting holes have the same measurements as the camera standoffs, so the same process can be applied to the IMU
- Positioning the IMU at the edge of the ball around the equator provides a convenient point of access for wires
- This can be seen in **Figure 4.2.2**

In SolidWorks, the workflow is:

1. Create a reference plane tangent to the sphere at the camera location.
2. Sketch the standoff pattern on this plane using the measured hole spacing.
3. Extrude the four cylinders inward toward the sphere’s center until they nearly touch the inner wall (leaving ~ 1 mm clearance).
4. Cut the internal pilot holes through the standoffs.
5. Cut a circular opening in the outer shell to match the camera lens shroud.

This retains the same measurements and workflow of the previous team, as we retained most parts of their design.

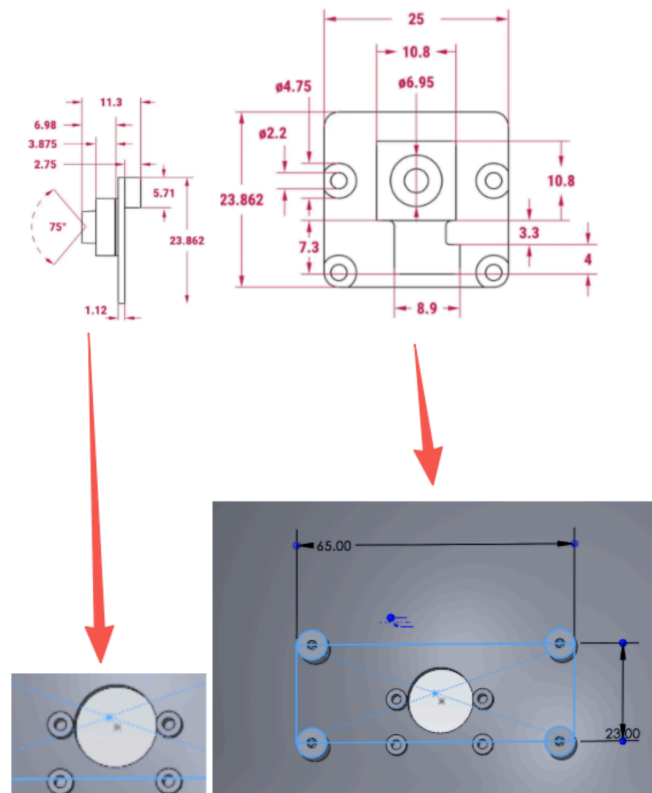


Figure 4.2.1: Translation of the Raspberry Pi camera module and Pi PCB dimensions into the SolidWorks mounting sketch. The top drawings show the manufacturer-provided mechanical dimensions of the camera module and its PCB, including the 4.75 mm mounting pads and 2.2 mm screw holes. Using these measurements, the bottom images define the corresponding bolt-circle and standoff locations on the spherical shell in SolidWorks, creating a 65 mm × 23 mm rectangular pattern of screw pillars that matches the actual board hole spacing.

During assembly, cameras are secured using M2.5 screws, and the CSI ribbon connects to the nearest Raspberry Pi Microcomputer.

4.2.4 Pi Mounting and Battery Support Standoffs

Below the equator, we designed a second set of internal mounts:

- A Pi mounting shelf – a rectangular boss ($\approx 66 \times 31$ mm) located on an internal tangent plane, extruded ~ 3 mm from the inner surface. Each Pi Zero 2 W rests on this shelf.
- The Pi4B receives its own mount on the shell with only one camera opening. The Pi4B is much bigger so the following accommodations must be made:
 - The Pi4B has its mounting holes spaced at 58mm x 49mm rather than the zero's 58mm x 23mm, so the standoffs must be spaced 13mm further on both the lower and upper ends.
 - The Pi4B also has additional ports to the side of the mounting holes that must be accounted for. Moving these standoffs to the opposite side. ~ 15 mm is sufficient.
 - The Pi4B must also rest further from the outside of the shell as it is larger in width and length. To accommodate we must raise the standoffs to ~ 10 mm higher. To support these longer standoffs, they should be thickened to increase the amount of stress they can handle. ~ 5 mm in radius is sufficient.

- These can be seen in **Figure 4.2.2**
- Four M2.5 standoffs with holes similar to the camera standoffs fasten each Pi. The pi boards are arranged such that the micro-SD card slot and USB ports remain minimally accessible when the ball is opened.
- The standoffs are positioned so that each Pi is roughly behind its corresponding camera, minimizing CSI ribbon length and cross-overs.

The Powerbank we used has the following dimensions:

- 3.6"L x 2.4"W x 0.5"Th (91.44mm x 60.96mm x 12.7mm)

To support powerbank in, we added supports that would hold the powerbank above the Raspberry Pi4B. This helps in the scenario where the powerbank would have otherwise fallen onto the Pi4B and possibly have damaged it or any of its wire connections. These should be spaced according to the powerbank's dimensions listed above. These supports should also be ~45mm tall to clear the Pi4B. These can be seen in **Figure 4.2.2**

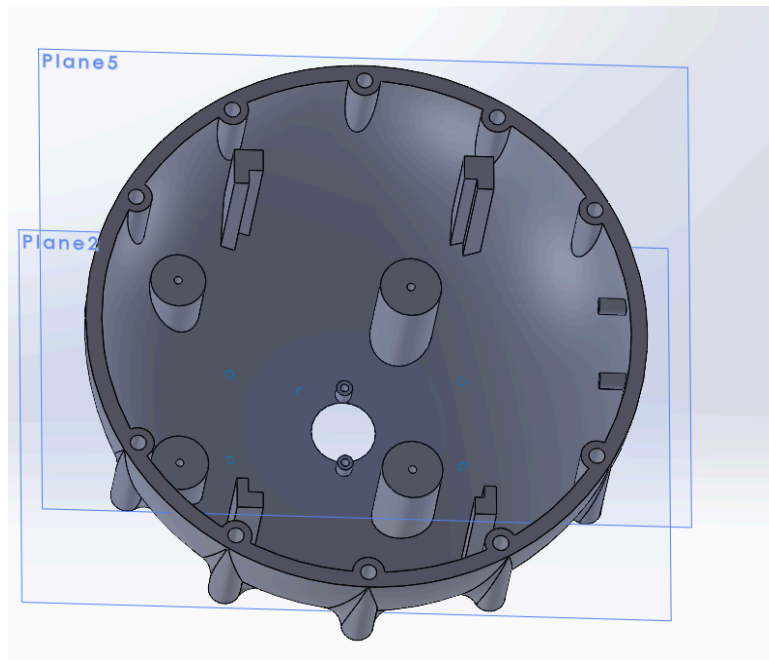


Figure 4.2.2: Final CAD file of updated shell design. This half accommodates the Raspberry Pi4B, the Adafruit ISM330DHCX IMU, and offers stands for the powerbank. It should be noted that the standoffs for the Raspberry Pi4B are noticeably thicker to better support and absorb impact.

4.2.5 3D-printing process

For reproducibility, we document the print settings used:

- Printer: Creality K1 Max (or similar Low Fidelity Machine at ≥ 300 mm build volume).
- Material: 1.75 mm ABS.
- Layer height: 0.2 mm for outer surfaces; 0.28 mm is acceptable for internal test prints.
- Nozzle diameter: 0.4 mm.
- Infill: 20–30 % gyroid or cubic, with 4–5 perimeter walls. More shells near the equator to stiffen the joint region.

- Print orientation: each half printed with the equator side down on the build plate. This maximizes strength in the radial direction and keeps camera surfaces clean.
- Bed temperature: 100 °C; nozzle temperature: 245–250 °C (tuned to filament).
- Enclosure: closed printer or external enclosure is strongly recommended to prevent ABS warping and layer separation.

This retains the same process done of the previous team, as we retain most of their design.

4.2.6 Mechanical Assembly Procedure

1. Prepare mechanical parts

- Clean 3D-printed halves; chase all pilot holes using a 2.3 mm drill to ensure smooth screw insertion.
- Tap camera and Pi standoffs if using machine screws, or use self-tapping M2.5 screws.

2. Install camera modules

- Attach each Raspberry Pi Camera 3 Wide to its standoffs using four M2.5 × 6 mm screws.
- Route each CSI ribbon inward through its slot. Secure cables with small adhesive clips or printed cable guides to avoid fatigue under impact.

3. Install Pi Zero boards

- Mount a Pi Zero 2 W onto each internal shelf of the half with two camera holes, aligned so the CSI connector faces its camera cable.
- Connect CSI ribbons, ensuring no severe bends and a bit of slack for shock absorption.
- Optionally add small foam pads between Pi and shell to damp vibrations.

4. Install Pi 4B board

- Mount the Pi 4B onto the half with the single camera hole with thicker standoffs, with the CSI connector connected to the camera below it and ensure there is no severe bend and a bit of slack for shock absorption.
- Optionally add small foam pads between Pi and shell to damp vibrations.

5. Connect Wires and Powerbank

- Using the Micro-USB power cables connected to the CMTPD board, connect each Pi Zero 2 W to the powerbank. Connect the Pi 4B to the powerbank with its own USB-C wire.
- To reduce wire clutter, wrap and arrange the wires around internal components and keep them bundled together.
- Seat the powerbank above its installed support beams and surround it with electronically safe packaging foam to reduce movement and vibrations

6. Close the shell

- Align alignment pins on the two halves.
- Route any remaining cables so they are not pinched in the seam.
- Fasten the equator screws (e.g., a ring of M3 screws spaced every 30–45°) and mark one side as “IMU” so that it is known which side the IMU’s position starts during initialization.

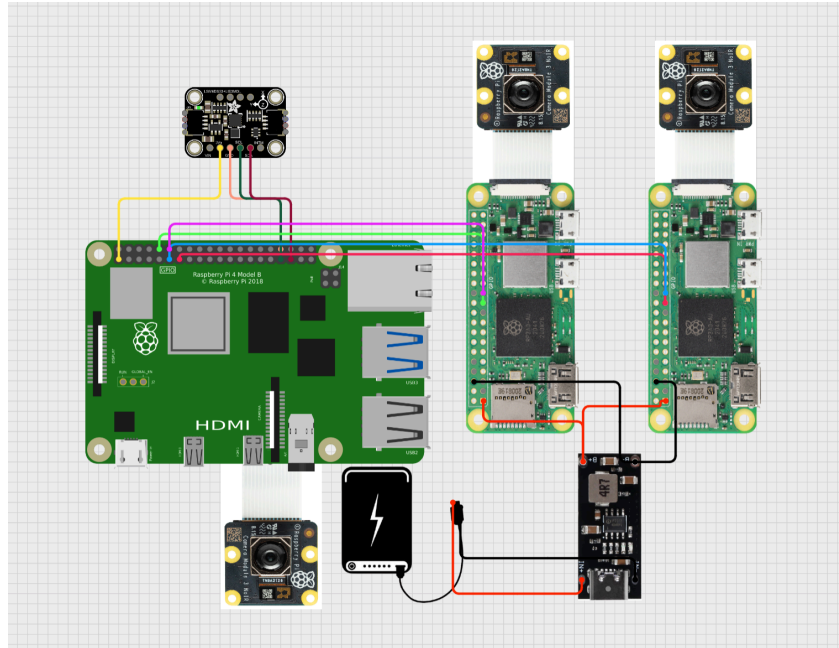


Figure 4.2.3: Final Schematic of hardware setup with connections.

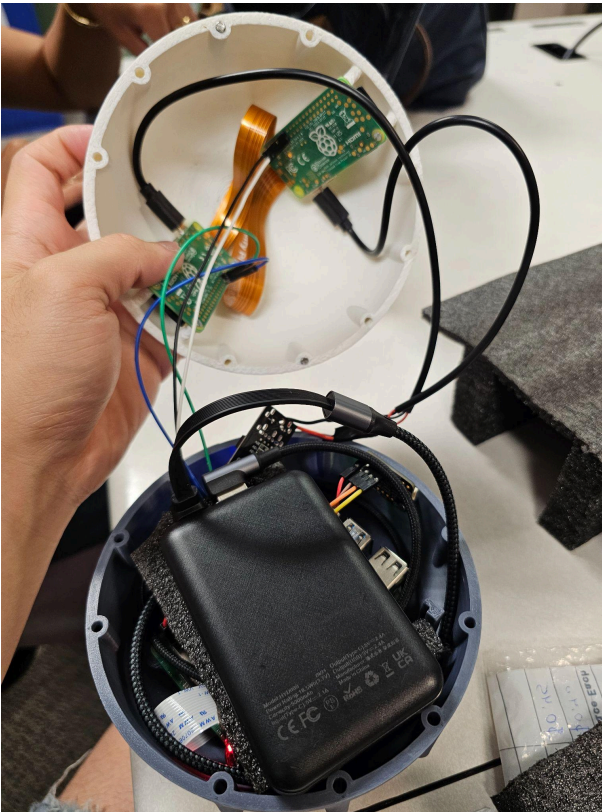


Figure 4.2.4: Bottom Half assembled while connected to Top



Figure 4.2.5: Bottom Half assembled without battery

4.3 Electrical

The electronics in the ball are kept modular and simple, using basic components available commercially for ease of use and replacement. The flow of electricity is as follows

1. 5W 2.4A Powerbank
2. Using two different wires:
 - a. A USB-A to USB-C wire directly connects to the Raspberry Pi 4B
 - b. A USB-C to USB-C wire directly connects to the CMTPD Board for power distribution
3. The CMTPD Board accepts a USB-C input and is directly soldered to two Micro-USB cables limited at 5V. These connect to both Raspberry Zero 2 W modules.

It should be noted that while the Raspberry Pi 4B has overvoltage protection, the Raspberry Pi Zero 2 W boards do not. While the CMTPD Board should maintain a 5V output, if for any reason the voltage manages to rise above 5V for a short period of time, it can damage the Raspberry Pi Zero 2 W boards.

4.4 Embedded Software

Each Raspberry Pi board runs a lightweight Linux system meant to process and upload camera data as fast as possible.

4.4.1 Base OS and configuration

1. Install the latest version of the Raspberry Pi OS Imager software from the official website.
2. Select Pi model
 - a. Select **Use Custom OS**
 - b. Search for **Raspberry Pi OS Lite (32-bit Debian Trixie)**
3. Configure username, password, initial Wi-Fi/Network connection, and enable SSH.

***When using a hotspot, it should be configured to a teammate that has an android phone due to the fact that an android hotspot setting allows the user to view devices connected to hotspot, and also its IP.**

*Each Pi is given a unique and differentiable host name (rpizero2w1, rpizero2w2, rpi4b)

4. Once the SD card has been successfully written, eject from device and insert into Pi.
5. Ensure that the configured Wi-Fi/Hotspot/Network is already turned on before powering the Pi on.
6. To connect to the Pi for headless mode:
 - a. Connect laptop or PC to configured hotspot/network.
 - b. Plug the Pi to the power source and wait for at least 2 minutes for it to fully boot.
 - c. Open Powershell and SSH to the Pi:

```
>> ssh raspberrypi@<raspberrypi's IP>
```

- d. Enter the configured password.

- e. Update the system:

```
>> sudo apt update && sudo apt upgrade -y
```

- f. To enable camera interface, run the command:

```
>> sudo raspi-config
```

- g. Navigate to **3 Interface Options - Configure connections to peripherals**
 - i. Enable the following: I4 SPI, I5 I2C
 - ii. Select Finish

- h. Upon completing these steps, the Pi is now successfully set up with camera interface enabled, with the next step being to set up Tailscale VPN Network to allow easy access regardless of what network the remote devices are connected to.

4.4.2 Tailscale VPN Network Setup

We use Tailscale to create a secure mesh network between the Pis and the server:

1. Install Tailscale (`curl https://tailscale.com/install.sh | sh`).
2. Authenticate each Pi to the team's Tailscale account and tag them appropriately.
3. Configure Tailscale to start automatically on boot.
4. Verify that each Pi can ping the server's Tailscale IP address even when connected to different physical Wi-Fi networks.

This setup allows deployment in any Wi-Fi environment that provides outbound internet, making it heavily convenient for microcontroller access regardless of what network the user device (laptop, PC) is on.

4.4.3 Camera Capture

Each camera runs our own camera client. This can be set up using the following process.

1. Clone the mast3r-camera-client repository onto the corresponding Raspberry Pi
2. Run the setup script with “`sudo bash setup.sh”` to install required dependencies
3. Start the service using “`sudo systemctl start mast3r-camera`”

The server will need to be configured to your server's IP address. Please consult the GitHub and [README](#) for more information.

The Raspberry Pi 4B will also run the gyroscope controller. Similarly clone the gyroscope_controller repository onto the Raspberry Pi 4B and use the camera_down_detector.py.

4.5 Server-side 3D Model Generation and Object Detection

The server is an Amazon Cloud Computing service with GPU processing. It receives images from each Pi, and uses them for 3D Model generation and object detection. If using a cloud server, ensure it has the proper CUDA version for your code.

4.5.1 Data input

Using Tailscale VPN Mesh Networking, we can establish a secure, encrypted tunnel between the ball and the server. Using this method allows us to connect the ball and server across different Wi-Fi networks. Using FastAPI, we send timestamped camera frames that are compressed and downsampled to save processing power. Images from the ball are stored in an “incoming_images” folder where they are used for both object detection and MAST3r-SLAM.

4.5.2 Object Detection Pipeline

The object detection subsystem operates as a server-side processing pipeline that analyzes incoming image data using the YOLO11s object detection model. The pipeline is designed to process images asynchronously as they are received from the Raspberry Pi capture nodes.

The incoming images are first stored in a designated directory (`incoming_images/`) by the image receiver API. The YOLO processing server continuously monitors this directory for newly added files. When a new image is detected, it is automatically loaded and passed to the YOLO11s model for inference.

During processing, the model analyzes each image and generates detection outputs consisting of bounding boxes, class labels, and confidence scores for all recognized objects within the frame. These detections are stored and used to generate annotated images with overlaid bounding boxes.

A filtering step is applied at the image level to reduce unnecessary output on the operator user interface. While all the images with detected objects are preserved in the backend results, only images containing at least one detected living object are forwarded to the web-based detection viewer. This filtering reduces visual clutter and prioritizes relevant scene information for the operator.

Processed images with detection overlays are saved to an output directory and made accessible to the viewer interface. The web viewer retrieves these images and displays them alongside detection metadata, including object class labels and associated confidence values.

Processing Steps:

1. Images are uploaded to the `incoming_images/` directory via the image receiver API.
2. The YOLO processing server monitors the directory for new files.
3. Detected images are processed using the YOLO11s model to generate bounding boxes and labels.
4. Images containing at least one living object are selected for display.
5. Annotated images are saved and served to the web viewer.

4.5.3 MAST3R-SLAM Pipeline

The MAST3R-SLAM subsystem is responsible for generating a 3D reconstruction of the environment using the images collected by the device. This pipeline operates on the same dataset as the object detection system, using images stored in the `incoming_images/` directory.

The system runs the `image_receiver_api.py` script in continuous mode using the `--continuous` flag. In this mode, the pipeline periodically checks for newly received images and incrementally updates the reconstruction model. This allows the system to refine the 3D model over time as additional images are captured and uploaded.

As part of the preprocessing, newly received images are converted into PNG format to ensure compatibility with the MAST3R-SLAM pipeline. These images are then processed to estimate camera poses, including position and orientation, by identifying and matching gesture points across multiple views. Using these estimated poses, the system reconstructs the spatial structure of the environment.

The reconstructed environment is further refined using Gaussian Splatting techniques to generate a dense and visually coherent 3D model. Once a reconstruction cycle is completed, the updated model is exported and made available to the web-based viewer. This process repeats at configurable time intervals, allowing the model to be continuously updated as new data becomes available.

While `image_receiver_api.py` runs in continuous mode:

1. New images in “`incoming_images`” are converted into png formats
2. New png versions of images are processed for model generation, interpolating points and estimating camera position and orientation based on anchor points in collected images
3. Once model generation is complete, the completed model is uploaded to the web viewer where it can be freely viewed by the user
4. This process repeats periodically based on configured wait time, effectively keeping the model on the viewer as up to date as possible

4.5.4 Web-based viewer

The system includes a web-based interface to provide intuitive control and visualization without requiring technical expertise. The interface is hosted on a central server and accessible via web browser over a secure Tailscale connection. The interface provides direct control over camera client on Raspberry Pi using only a couple of buttons that can be pressed to start and stop capture. When activated, the interface sends an http request to the API endpoint, which triggers the corresponding action on Raspberry Pi.

To provide real-time feedback, the interface includes a system status indicator that reflects the current state of the pipeline:

- Loading
- Ready
- Processing

The interface displays outputs for both 3D reconstruction (via Master slam and Gaussian splatting pipeline) and object detection (via YOLO annotations overlaid on captured images)

4.6 Testing

A full end-to-end test to simulate real use:

1. Use a room with furniture, doorways, obstacles, people (if possible)
2. Connect battery to device and wait for it to join network
3. Operator has Viewer open and presses “start capture”
4. Tester rolls ball into the room from outside the doorway
5. Operator monitors incoming images being sent in
6. When enough time has passed that satisfies operator, operator presses “stop capture”
7. Model generates after 45-70 seconds
8. Once the model is ready, operator uses Viewer to identify
 - a. Room layout
 - b. Primary obstacles
9. On top right corner, operator presses “detections” to view all images with object detection annotations

4.7 Future Considerations

4.7.1 Software

3D Gaussian Splatting requires a large number of images to be captured from different angles in order to achieve an accurate model. When our device is rolled, there are an insufficient number of images and angles. Future work should explore alternate reconstruction methods that can handle this issue.

Another area of improvement is the image capture and transmission pipeline. The current approach relies on discrete image capture followed by http-based uploads, which introduces latency and limits overall throughput to the server. A more efficient method could improve data transfer rates and reduce delays in processing. Increasing throughput would improve both reconstruction accuracy and real time responsiveness.

The system could also be improved by enabling a live camera feed once the ball is done rolling. Once the ball is at rest, the Raspberry Pi could switch from burst image capture to continuous video streaming, allowing the operator to observe the environment in real time. This would provide immediate situational awareness before 3D reconstruction is complete.

The user interface and control architecture could also be improved. Currently, the start/stop controls in the web interface are linked with camera initialization, leading to increased startup latency. Decoupling the

start/stop capture buttons on the Viewer could reduce startup time and allow near instant activation of capture when operator issues command.

4.7.2 Hardware

Due to budget constraints, using multiple independent microcontrollers ensures true simultaneous and parallel processing in order to improve capture rate. A multi-cam adapter operates cameras sequentially rather than simultaneously. A true simultaneous setup such as a Quadrascopic Stereo Camera or a Synchronized Stereo Camera would be too expensive (\$225-240 per unit).

For the power supply, ensuring that all wiring and cables fit well within the enclosure is critical for reliability and durability. One solution used is a right angle charging cables, which reduced strain on connectors and improved internal cable routing.

The choice of operating system of each microcontroller must align with the software libraries and dependencies required for the system pipeline. For example, object detection frameworks, such as Ultraanalytics, are not compatible with older operating systems such as Raspberry Pi OS Bullseye. Future implementations should standardize on operating systems that support current libraries and toolchains, ensuring compatibility with Slam, object detection, or other processing tasks.

4.7.3 Shell

The current shell could be improved by modifying it to be more spherical. A rounder design would ensure smoother and more predictable rolling. Additionally, reducing the number of screw threadings for the enclosure would reduce the maintenance time for assembling and disassembling. Expanding the internal volume would also be helpful to better accommodate wiring and cable connections.

The shell would also benefit from a more secure mounting solution for power supply to ensure it remains fixed and protected during operation. A dedicated compartment would prevent movement that may lead to disconnections or damage. It would also be helpful to incorporate easy access mechanisms (like a pin hole or a latch) to allow users to replace the battery without disassembling the shell.

5 Experimental results (5-10 pages; 20 points)

This section is also very important, as it shows whether or not your project worked. You should provide:

- A description of the tests you performed to validate the system,
- Numerical results of these tests,
- A thorough analysis and discussion of these results, and
- A collection of pictures of your final system

5.1 Physical design

We used a scale and measuring tape to verify the weight and diameter of the ball was within the limit we set for it (7 inch for diameter and 1 lb for weight). This is to ensure that the ball is the perfect size and weight to easily roll the ball. The diameter of our ball ended up being 5.45 inches and the weight of the ball with all components ended up being 0.966 pounds (438 grams).

Measured



5.2 Multi-Camera Capture

We showed that the `incoming_images` folder is initially empty. We start capture while the ball is stationary as it is taking pictures. This allowed us to show the performance of the cameras without taking into account motion. We open the `incoming_images` folder and show that it has populated shortly after initialization, confirming that the cameras were actively capturing images and saving the data. The images clearly showed different perspectives of the same environment, demonstrating that the three cameras were properly oriented at distinct angles and functioning simultaneously. Timestamps are indicated to evaluate how fast the capture rate was.

```

(.yolo_venv) PS C:\Users\wu-wi\Documents\TAMU\SP26\CSCE483\code\MAST3R-SLAM-API> ls .\incoming_images\

Directory: C:\Users\wu-wi\Documents\TAMU\SP26\CSCE483\code\MAST3R-SLAM-API\incoming_images

Mode                LastWriteTime         Length Name
----                -
-a----            4/26/2026 12:27 PM        22450 raspi_cam_20260426_122735_949855.jpg
-a----            4/26/2026 12:27 PM        22446 raspi_cam_20260426_122736_290775.jpg
-a----            4/26/2026 12:27 PM        14123 raspi_cam_20260426_122736_380034.jpg
-a----            4/26/2026 12:27 PM        22440 raspi_cam_20260426_122736_640740.jpg
-a----            4/26/2026 12:27 PM        18061 raspi_cam_20260426_122736_867831.jpg
-a----            4/26/2026 12:27 PM        22413 raspi_cam_20260426_122736_977058.jpg
-a----            4/26/2026 12:27 PM        14016 raspi_cam_20260426_122737_012572.jpg
-a----            4/26/2026 12:27 PM        22475 raspi_cam_20260426_122737_313544.jpg
-a----            4/26/2026 12:27 PM        17037 raspi_cam_20260426_122737_498094.jpg
-a----            4/26/2026 12:27 PM        22430 raspi_cam_20260426_122737_649462.jpg
-a----            4/26/2026 12:27 PM        14013 raspi_cam_20260426_122737_650118.jpg
-a----            4/26/2026 12:27 PM        22441 raspi_cam_20260426_122737_996216.jpg
-a----            4/26/2026 12:27 PM        16918 raspi_cam_20260426_122738_130813.jpg
-a----            4/26/2026 12:27 PM        13997 raspi_cam_20260426_122738_283832.jpg
-a----            4/26/2026 12:27 PM        22433 raspi_cam_20260426_122738_336642.jpg
-a----            4/26/2026 12:27 PM        22437 raspi_cam_20260426_122738_674444.jpg
-a----            4/26/2026 12:27 PM        17213 raspi_cam_20260426_122738_762695.jpg
-a----            4/26/2026 12:27 PM        13949 raspi_cam_20260426_122738_919146.jpg
-a----            4/26/2026 12:27 PM        22445 raspi_cam_20260426_122739_012803.jpg
-a----            4/26/2026 12:27 PM        22410 raspi_cam_20260426_122739_352287.jpg
-a----            4/26/2026 12:27 PM        16873 raspi_cam_20260426_122739_390300.jpg
-a----            4/26/2026 12:27 PM        14017 raspi_cam_20260426_122739_547620.jpg
-a----            4/26/2026 12:27 PM        22448 raspi_cam_20260426_122739_697372.jpg

```

Figure 5.2: Console Log of the incoming images

Table 2: Image Capture Timing and Performance Summary

First image timestamp	12:27:35.95
Last image timestamp	12:27:39.70
Duration	3.75s
Total image count	23
Frames per second	6.13

5.3 Gyroscope and Camera cutting

We ran 2 different tests to verify the gyroscope and camera-cutting functionality: one where the ball is handheld and another test where the ball is rolled as it would be in the real use-case. This test is meant to determine if the downward images can correctly be identified with the gyroscope. For handheld, the ball was manually positioned so that each camera, one at a time, was oriented downward toward the ground. During image capture, we monitored the terminal output to verify whether the system correctly identified the active downward-facing camera. This process was repeated for all three cameras to ensure

consistent behavior across the full camera array. The results showed that, in each case, the terminal correctly indicated when a camera was facing downward, confirming that the IMU-based orientation detection was functioning as intended in a controlled environment.

For rolling, the ball was placed on the ground, and image capture was initiated before rolling the ball across a short distance. This test introduced real-world motion and rotation, allowing us to evaluate the system under more realistic deployment conditions. After stopping the capture process, we examined the contents of the incoming_images folder. Each image was opened and visually inspected to determine whether it was facing downward. We then compared this observation with the corresponding filename metadata, which encodes orientation information.

As a result, downward detection reduces downward images by 40% without limiting useful data.

Table 5.3: Orientation Detection Performance Metrics

Metric	Description	Count
True Positive (TP)	Correctly detects the downward-facing orientation	2
False Positive (FP)	Incorrectly labels a non-downward orientation as downward	0
False Negative (FN)	Fails to detect when device is actually facing downward	3
True Negative (TN)	Correctly identifies non-downward orientations	21

5.4 SLAM Modeling and YOLO Object Detection

Using only one camera, we start capturing and have one person walk around a room holding the ball to take images. We only had one camera on to reduce the complexity of multi-camera inputs. Slow movement allows for better image overlap and improves the accuracy of the reconstruction process. When done capturing, the 3D model will show up on the viewer. Then, we open the object detections indicated by a button to see all of the images the camera took with object detection annotations.

As a result, the 3D model was able to get a good amount of the room if the person is slowly walking around. The reconstructed model was able to capture a good amount of tables, people at the tables, objects on the tables, and even some of the walls. Object detection was able to correctly identify people. However, it can sometimes have some inaccuracies when detecting inanimate objects such as mixing up TVs, doors, & monitors as well as chairs & couches due to how similar they look.



Figure 5.4.1: SLAM Modeling

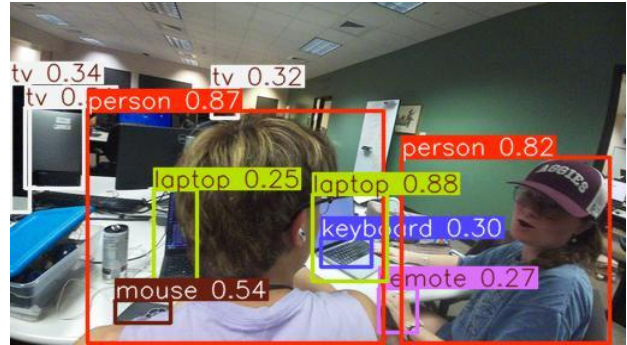


Figure 5.4.2: YOLO Object Detection

Table 5.4: Master SLAM Timing Measurements

Events	Time Elapsed (s)
GUI button pressed, First image captured	6
Image capture duration	0.15
First image upload -> Server received image	0.2
Server receives first image -> server receives enough images to generate model	10
Model generation duration	45-70
Client receives and parses model	3
Total	~1:05-1:30 minutes

5.5 Camera Performance in a low-light environment

Because we did not have access to anything that could mimic a smoky environment, we were only able to test in low light conditions. We used the same testing procedure as in the previous test: A user walked slowly around the room while holding the ball and capturing images using a single camera. However, for this test, the room lights were turned off, leaving only a small amount of light entering from outside the room. This created a high-contrast environment with both dimly lit and near-dark regions.

After completing image capture, the reconstruction pipeline was executed, and the resulting 3D model was displayed in the viewer. Object detection results were then examined through the annotation interface. As a result, the 3D model only included portions of the room that were illuminated by the incoming light, while areas in complete darkness were not reconstructed. This indicates that the reconstruction algorithm relies heavily on visible features and sufficient lighting to extract meaningful spatial information. Object detection was able to successfully identify objects in the room despite being in dark regions.

This test showed a clear limitation that the system requires adequate lighting in order to generate an accurate 3D model.

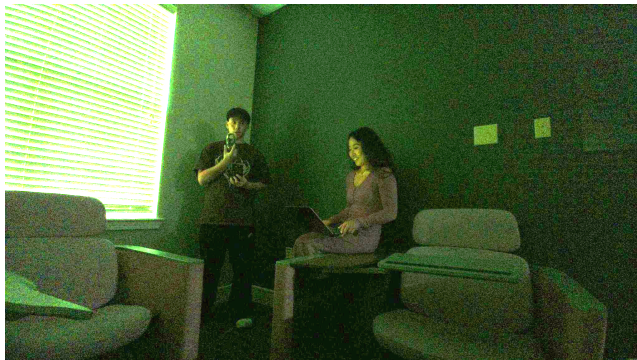


Figure 5.5.1: Adjusted Contrast & Brightness for verification of correct detections

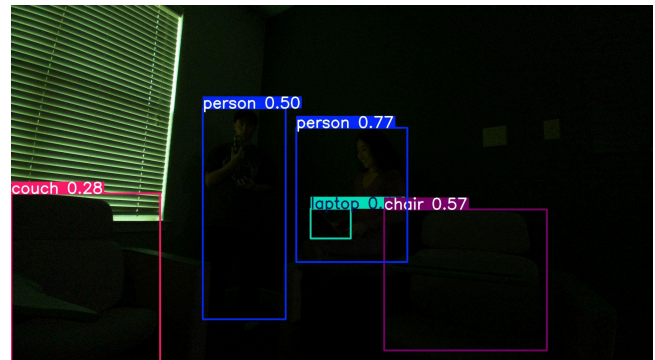


Figure 5.5.2: Original Image with Detections

5.6 End-to-End System Functionality

Evaluating the overall performance of the system, we conducted a test that combined all major subsystems, including image capture, camera orientation handling, 3D reconstruction, and object

detection, under realistic deployment conditions. The goal of this test was to simulate how the system would perform in an actual use case, where the ball is deployed by rolling it into an environment with people present. We started capture and then rolled the ball in a room with one person inside. This introduced rapid motion, varying orientations, and non-uniform image capture intervals. After the rolling motion was complete, image capture was stopped, and the collected data was processed through the reconstruction pipeline. The resulting 3D model was then viewed using the system's visualization interface. Object detection results were also examined through the annotated image viewer.

As a result, the 3D model contained only a limited representation of the room, with large portions either missing or poorly defined. This was primarily due to the fact that images captured during rolling were relatively sparse and taken from rapidly changing perspectives, resulting in insufficient overlap between consecutive frames. The object detection continued to perform effectively. Despite the reduced number of usable images and the motion-induced variability, the system was still able to accurately identify key objects within the captured frames. This demonstrates that object detection is more robust to motion and sparse data compared to the 3D reconstruction process.

Overall, this test shows that the system was unable to create a 3D reconstruction of a room from rolling the ball into that room. In order to get an accurate result of the model, the ball must be handheld and slowly moved around the room, which ultimately defeats the purpose of this project.

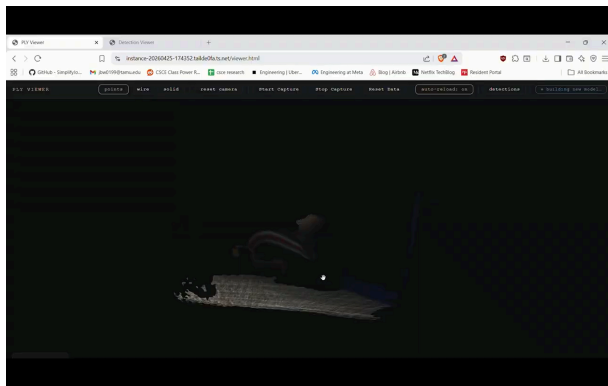


Figure 5.6.1: 3D Reconstruction

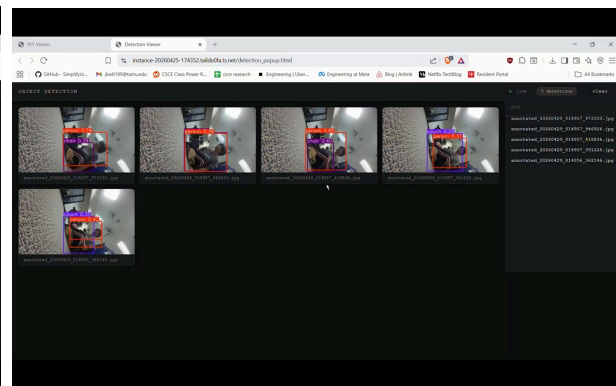


Figure 5.6.2: Detections Tab with Object Detections

6 User's Manuals (5-10 pages; 20 points)

This document should contain detailed (1) hardware installation, (2) software installation, and (3) operation instructions for the users of your product(s). It should be aimed at the “average user” of your system (please identify the characteristics of such user) and should not require that the reader be an engineering professional.

6.1 Introduction

The Emergency Response Mapping Sphere is a tactical device designed to provide situational awareness before entering unknown indoor environments. After being deployed into the room, the device automatically captures a 360 view, streams encrypted data, and creates a 3D model of the space. This tool helps reduce risk for first responders operating in unknown rooms, collapsed buildings, and other dangerous structures. The system requires no engineering knowledge and relies on a simple web-based interface.

6.2 Safety information

Warnings:

- Avoid heat, fires, and high temperature environments unless using a certified fire resistant shell
- Roll ball only into rooms, not directly at people
- Use a controlled roll to avoid bouncing (roll it like a bowling ball)
- Keep away from water and corrosive liquids

Safety operation practices:

- Check surroundings before rolling
- Maintain a secure grip when preparing to roll
- Do not deploy near explosives or chemicals

6.3 Intended User Profile

This product is intended for first responders such as firefighters and police officers, not civilian use.

Assumed skill level:

- Comfortable using a laptop
- No requirement for engineering or programming skills

Operating conditions:

- Unknown rooms
- Collapsed structures
- Disaster areas
- Debris zones and low visibility spaces

6.4 Package contents

Package should include

- 1 Emergency Response Mapping Sphere
- 1 charging brick with power interface
- 1 USB-C charging cable
- User manual

6.5 System overview

Once powered on and deployed,

1. Device boots in 45 seconds
2. Must manually press Start/Stop Capture on Viewer on a web browser
3. Master slam and Gaussian Splat 3D model is created, as well as Object Detection annotations
4. Model is shown on Viewer
5. Allow pan, zoom, and rotate interaction
6. Object detection annotation can be viewed on Viewer by pressing “detections” button

Connection uses:

1. WiFi with VPN encryption
2. Local server software
3. Manual device ID pairing

6.6 Hardware Installation

Physical specifications

- Shape: sphere
- Diameter: 5.45 inches (7 in max set by us)
- Weight: 0.996 pounds (1 lb max set by us)
- Material: ABS filament (3D printed/CAD design)

Internal components:

- 1 Raspberry Pi 4B microcomputer
- 2 Raspberry Pi Zero 2 W microcomputer
- 3 NoIR (No IR filter) Camera modules
- Portable USB battery bank
- Gyroscope module (Camera-cutting)
- CMTPD Board (Power distribution)
- Foam cushion

Durability:

- Designed for short-ranged tactical rolls
- Avoid excessive force to prevent bounces or other unpredictable behavior

6.7 Charging and Power

Charging instructions

1. Connect USB-C cable to charging block
2. Insert cable into power outlet
3. Charge until full (3-4 hours)
4. Put charging block back into appropriate place and screw the shell back together

Powering On

- Device powers when battery is connected
- No power button
- Boot time: ~45 seconds

Powering Off: disconnect battery cable

6.8 Software Installation

System Requirements

- Laptop running Windows/macOS
- Wi-Fi capability
- Local Administrator permissions

Installation Steps

1. Download Local Server Program
2. Run the Installer and follow the prompts
3. Start the program
4. The browser interface opens automatically

6.9 Device Pairing

1. Open the Viewer on web browser
2. Ssh into Pis based on its Tailscale IP address
3. Enter login credentials

6.10 Operation of Product

Step-by-Step Deployment

1. Charge the device fully
2. Connect the battery cable (device turns on)
3. Wait ~45 seconds for boot
4. Open web UI
5. Press button “start capture” on Viewer, which initializes camera client on Pi
6. Roll device like a bowling ball into target room
7. Press “stop capture” when ball stops rolling
8. View the model on the laptop

Viewer

- Model displayed after ~45-70 seconds
- Use the mouse to:
 - Rotate view
 - Zoom
 - Pan
- To view object detection annotations: press the “detections” button at top right corner. This will open up all the object detection images in a new tab.

Connection uses:

- Wifi with VPN encryption
- Local server software
- Manual device ID pairing

6.11 Interpolating 3D Model

The device uses Master Slam and Gaussian Splat modeling.

Viewer features

1. Start/Stop capture button
2. 3D space visualization
3. Object detection annotations

Lighting conditions

- Low light or reflective surfaces may reduce clarity
- Disconnect if storing long term

6.12 Maintenance

Shell inspection

After heavy impact

- Check shell for cracks
- Replace shell if damaged

Cleaning

- Wipe with dry cloth
- Avoid solvents
- Do not submerge

Storage

- Room temperature storage
- Avoid sunlight/heat
- Charge fully before storing

Battery Care

- Use only included charger
- Disconnect if storing long term

6.13 Technical specifications

Table 6.13: System Technical Specifications

Feature	Specification
Diameter	5.45 in
Weight	0.996 lb
Shell material	ABS filament
Boot time	~45 seconds
Reconstruction	Master slam and gaussian splatting
UI	Local web interface
Encryption	VPN

6.14 Troubleshooting

Table 6.14: Common Issues and Troubleshooting Solutions

Problem	Cause	Solution
Device hot	High temp	Remove battery, let it cool
No connection	Pi not connected	Reboot the Pi
Device not powering on	Battery is dead	Recharge battery
Poor 3D model	Low light or fast motion	Improve lighting or move slower

Camera not working	Loose camera ribbon cable	Reattach the ribbon cable
Images not uploading	Network failure	Restart device and verify server connection
3D model not created yet	Model has not finished generating	Wait 45-70 seconds

7 Course debriefing (2-4 pages; 10 points)

This document should contain the group's collective answers to the following issues.

Provide a thorough discussion of your *team management* style. If you were to do the project again, what would you do the same, what would you do differently?

Are there any particular *safety* and/or *ethical* concerns with your product(s)? What steps did your group take to ensure these concerns were addressed? Are there any additional steps you would have taken if you were to do the project again?

Did you test your product(s)? Do they work as proposed? Can you think of any relevant situations in which you haven't tested your product(s)? If you were to do this project again, what additional *verification* and *testing* procedures might you add?

Similar to the predecessor team, our management style began somewhat loosely structured at the start of the semester. The key difference however, was that we established a team lead whose primary responsibility was to evaluate each member's strengths, weaknesses, and preferred areas of work before delegating responsibilities. This allowed us to split the team into two primary subteams. The hardware subteam focused on physical design and electronics integration, and the software subteam was responsible for embedded capture pipeline, networking, and server-side reconstruction. The intent was to ensure that each member was working in the area where they could contribute most effectively and that both subsystems could progress concurrently.

In practice, this structure worked well early in the semester, with everyone contributing to the improved hardware and software system designs. In addition, both subteams also had a primary lead to delegate further tasks and gauge cross-subteam dependencies. However, as the project progressed, the subteam boundaries became less defined. Some members found themselves contributing to both hardware and software tasks simultaneously or having to pick up other subteam member's tasks, creating an uneven distribution of workload that was not always recognized or rebalanced in time. Moreover, there was a consistent pattern of significant prerequisite tasks not being completed on time, which introduced substantial downstream delays in testing and key implementation milestones such as MAST3R-SLAM integration. A particularly costly example was the 3D-printed enclosure not being designed and printed until very late into the project, which also required lab clearance and printing time. This directly cost the team approximately two weeks of potential testing and integration time. There were also tasks that showed no demonstrable progress both inside and outside of the weekly team meeting throughout the semester. The issue was compounded by the fact that the team did not establish formal accountability mechanisms early enough, so by the time the lack of contribution and uneven task completion became apparent, there was limited time and leverage to correct it.

If we were to redo the project again, we would keep the practice of appointing a team lead and splitting into subteams early, as this proved to be a genuine advantage over a fully informal approach. What we would do differently is implement strict, formal progress tracking from the very beginning of the semester, defining deliverables per member at each weekly checkpoint and an explicit expectation that incomplete or absent progress would be called out immediately rather than quietly absorbed by the rest of the team. Relying on teammates to self-manage without enforcement proved costly, and a future team in

this position should not hesitate to escalate non-contribution issues early, whether to the team lead, instructor, TA, or all. Establishing this level of accountability from the beginning would have allowed us to address participation issues before they escalate into consistent critical delays on hardware that the entire system depended on.

Our project involved working with multiple electronic components and power sources, and the most significant safety concern we countered was also related to the power supply. During extensive testing, we observed on multiple occasions that the battery overheated significantly under load. Because a single battery was responsible for powering all three microcontrollers, with two of them drawing current simultaneously through a shared power distribution board connected directly to the battery, the thermal load exceeded what the battery could safely sustain during prolonged operation. The consequences of this were significant: the overheating caused SD card corruption across all three Pi microcontrollers, requiring each one to be fully reimaged before testing could resume. There were also frequent difficulties of the Pi attempting connection or sustaining initial connection to the network. While no fire occurred, the temperature reached during these incidents were concerning enough to treat it as a genuine fire safety hazard.

This introduces a clear ethical concern as well. A device intended for deployment in life-critical first responder scenarios carries an elevated standard of reliability. A battery that overheats under normal operating conditions does not only risk damaging the device itself but also introduces the possibility of a fire hazard in the very environments the device is intended to be thrown into, which could directly endanger the responders relying on it. Releasing or deploying such a product with a known thermal risk without resolving it would be ethically unacceptable. To address this concern, we immediately removed the battery and replaced it with a stronger battery to be used during extensive testing and did research on what went wrong. If we were to do this project again, we would conduct thorough thermal and power testing under realistic load conditions much earlier in the development cycle before any extended integration testing begins. We would also enforce a strict component vetting process to ensure that any battery used is rated for the combined sustained load of all connected components, with appropriate safety margins built in. One practical application would be to simulate the full hardware system using circuit simulation software prior to physical assembly and ordering, in order to gauge power consumption, validate component ratings, and verify overall specs before committing to a physical build.

We performed testing across all major functional components of the system, and while the core features operated as implemented, not all results met the original performance requirements established at the start of the project, particularly in the context of a real first-response deployment scenario. Object detection performed reliably and exceeded expectations, achieving a high rate of success across all demonstration trials, correctly identifying people even under partial occlusion, detecting small body parts such as legs, and functioning effectively in low-light environments. Camera-cutting logic driven by the gyroscope functioned correctly under slower, stable motion, but became unreliable at higher speeds or first-response scenarios due to consecutive frames sharing too few overlapping features, contributing to inaccurate model generations. The end-to-end pipeline latency, measured from the ball rolling to serving the 3D model, failed significantly short of the sub-30 second target. During demos, the pipeline averaged over a minute to complete in all scenarios, tossing or rolling the device into a well-lit or low-visibility room. However, we were unable to test in scenarios that involved smoke-filled rooms, extreme temperatures, or just throwing the ball due to how fragile the internal components were. After investigation, we determined that the 3D reconstruction step alone from MAST3R-SLAM, is an irreducible computational bottleneck that cannot be accelerated below roughly 45 seconds given our own hardware setup. This is a fundamental system limitation and future iterations would need to evaluate lighter-weight reconstruction methods or more power dedicated hardware to address it. Other system limitations identified through testing include the server's data ingestion throughput being insufficient to keep pace with the full frame rate of the three Pis simultaneously, and the inability to check battery status without physically opening the shell. If we were to do this project again or start back earlier, we would

introduce more rigorous performance benchmarking earlier in the development cycle to identify the latency and reconstruction quality gaps. Testing the system under more realistic deployment conditions such as actual throws, varied lighting, and multi-room environments, would also be a priority in future iterations.

8 Budgets (2-4 pages; 5 points)

Summarize the costs of the prototype, including parts list with sources, and any support tools required (with costs.) Provide (1) detailed cost breakdowns, (2) a short narrative summarizing the costs, and (3) a comparison of these costs with the proposed budget and with your estimate of the cost in mass production.

8.1 Detailed Cost Breakdown

The total prototype cost for the emergency response mapping sphere is approximately \$249.95. **Table 8.1** summarizes the complete bill of materials, including all major hardware components and supporting materials required for system assembly.

Table 8.1: A parts list of materials that are needed to build the design.

	Amount	Cost per Unit (\$)	Total Amount (\$)
Raspberry Pi 4B, 2GB RAM	1	59.50	59.50
Raspberry Pi Zero 2W 512MB RAM	2	15.00	30.00
Raspberry Pi Camera Module 3 Wide NoIR	3	38.50	115.50
Adafruit ST ISM330DHCX 6-DOF IMU	1	19.95	19.95
Power Bank	1	9	9
3D-printer filament (ABS, 1 kg)	1	~6	~6
Misc. fasteners, heat-shrink, etc.	-	~10	~10
Total Cost			249.95

Total Estimated Cost: \$249.95

8.2 Cost Narrative

The primary cost driver of the system is the multi-camera setup required for 3D reconstruction. The Raspberry Pi Camera Module 3 Wide NoIR units account for the largest portion of the budget due to the need for three synchronized imaging nodes. The computing hardware, consisting of one Raspberry Pi 4B

and two Raspberry Pi Zero 2 W units, is the second major cost component and supports distributed image capture across the device.

This architecture replaced the original multiplexer-based design, which increased hardware cost but improved performance by enabling parallel image capture. Additional costs include the IMU for orientation tracking, a power bank for portable operation, and low-cost structural materials such as 3D printing filament and fasteners.

Overall, the system was implemented within a constrained prototype budget while meeting the required functionality for imaging, transmission, and processing.

8.3 Budget Comparison and Mass Production Estima

The system was designed under a budget constraint of \$250 or less, which served as the primary cost target throughout development. The final prototype cost of \$249.95 meets this requirement while still satisfying all functional requirements.

The largest contributor to cost is the multi-camera configuration required for 3D reconstruction, followed by the distributed computing setup using multiple Raspberry Pi devices. The transition from a multiplexer-based design to independent processing nodes increased hardware costs but improved system performance by enabling parallel image capture.

In a mass production scenario, the cost is expected to decrease due to bulk purchasing and hardware optimization. The estimated production cost per unit is approximately \$180–\$220, primarily driven by reduced pricing of camera modules and embedded computing components at scale.

9 Appendices

Here you can include any additional information not already in the implementation notes (e.g., detailed circuit schematics, class hierarchy) that would help replicate or maintain your implementation.

10 References

Here you acknowledge all the documents that you used as references throughout the text. Please use the IEEE reference format.

<https://ieeauthorcenter.ieee.org/wp-content/uploads/IEEE-Reference-Guide.pdf>